

Mindous.ai Agent Execution Engine - Technical Architecture

Executive Summary

This document provides a comprehensive technical blueprint for implementing the autonomous agent execution engine in Mindous.ai. It builds upon the existing infrastructure and outlines the architecture needed to enable users to prompt the agent to build applications, watch them being built in real-time, and test them directly in the chat thread - similar to Abacus DeepAgent.

Document Version: 1.0

Last Updated: November 17, 2025

Status: Planning Phase

Table of Contents

1. [Current State Assessment](#)
 2. [Architecture Overview](#)
 3. [Agent Execution Engine](#)
 4. [Build System Architecture](#)
 5. [Deployment & Preview System](#)
 6. [Database Schema Extensions](#)
 7. [API Route Structure](#)
 8. [WebSocket Integration](#)
 9. [Tool Framework Architecture](#)
 10. [Frontend Integration Points](#)
 11. [File & Artifact Management](#)
 12. [Implementation Roadmap](#)
 13. [Security & Permissions](#)
 14. [Performance & Scalability](#)
-

1. Current State Assessment

1.1 Existing Infrastructure

Database Schemas

- **Agents:** Agent types, capabilities, configurations
- **Tasks:** Hierarchical task structure with parent-child relationships
- **Executions:** Task execution tracking with status and metrics
- **Tools:** Tool registry with Docker container specs
- **Runs:** Overall execution run tracking
- **Progress Streams:** Real-time event streaming infrastructure

- **LLM Routing:** Multi-provider LLM routing with circuit breakers

API Routes

- `/api/tasks/decompose` - Task decomposition using LLM
- `/api/runs/create` - Create execution runs
- `/api/streams/runs/[runId]` - Server-Sent Events for progress
- `/api/tools/execute` - Tool execution endpoint
- `/api/chat/message` - Chat message handling

Services

- **Planning Service:** LLM-powered task decomposition
- **Tool Execution Service:** Tool orchestration and execution
- **LLM Router:** Multi-provider routing with caching
- **Tool Registry:** Tool management and validation

Frontend Components

- **Chat Interface:** Basic chat UI
- **Run Progress Panel:** Real-time progress visualization
- **Task Cards:** Hierarchical task breakdown UI
- **Tool Usage Panel:** Tool execution visualization

Environment Configuration

```
# LLM Providers
OPENAI_API_KEY=sk-proj-...
ANTHROPIC_API_KEY=sk-ant-...
ABACUSAI_API_KEY=s2_9537ddcd077146c4be92cb46d87a07e7

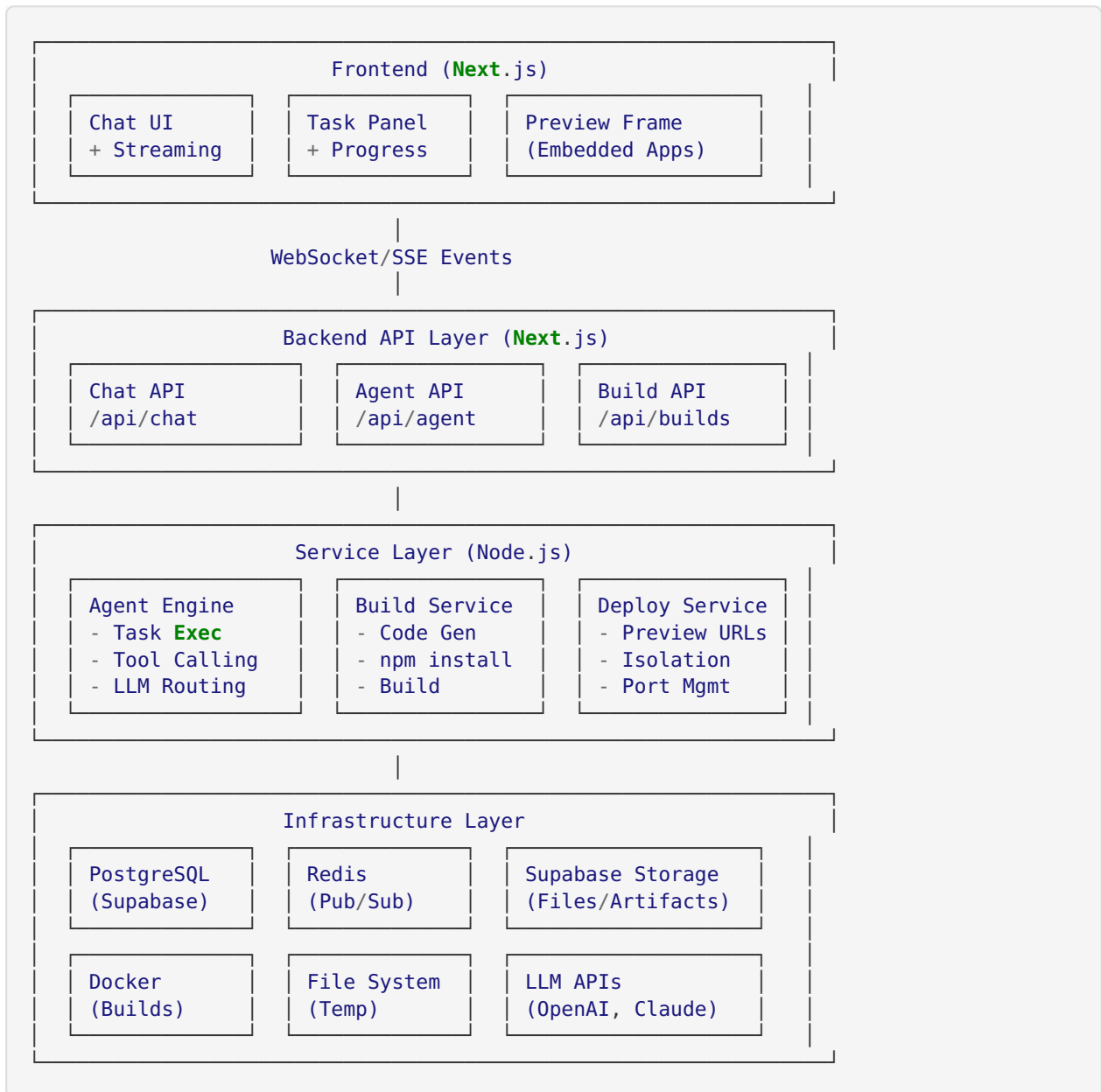
# Database
DATABASE_URL=postgresql://...
NEXT_PUBLIC_SUPABASE_URL=https://...
```

1.2 What's Missing

- ✗ **Agent Execution Engine:** Core autonomous execution loop
 - ✗ **Build System:** Next.js/React app generation and building
 - ✗ **Code Generation:** LLM-powered code creation
 - ✗ **Preview Deployment:** Isolated app hosting for testing
 - ✗ **File Management:** Artifact storage and retrieval
 - ✗ **Build Orchestration:** Docker-based build pipeline
 - ✗ **Preview Embedding:** Iframe integration in chat
 - ✗ **Code Streaming:** Real-time code generation display
-

2. Architecture Overview

2.1 High-Level System Design



2.2 Core Components

Component	Location	Purpose
Agent Engine	/lib/agents/execution-engine.ts	Core autonomous execution loop
Build Service	/lib/services/build-service.ts	Next.js app generation & building
Deploy Service	/lib/services/deploy-service.ts	Preview deployment & hosting
Code Generator	/lib/services/code-generator.ts	LLM-powered code creation
File Manager	/lib/services/file-manager.ts	Artifact storage & retrieval
Preview Manager	/lib/services/preview-manager.ts	Preview URL & port management

3. Agent Execution Engine

3.1 Architecture

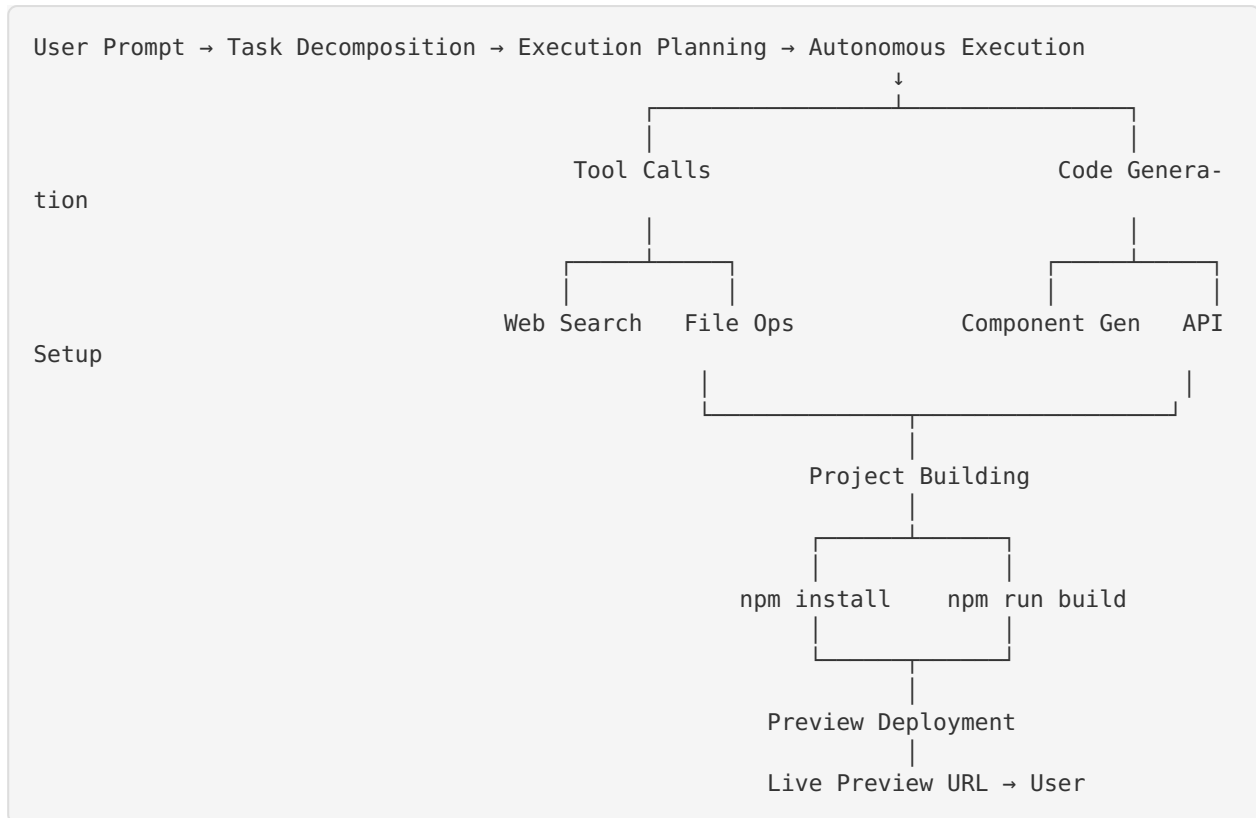
The agent execution engine is the brain of the system. It orchestrates the entire execution flow from task decomposition to final delivery.

Location: /lib/agents/execution-engine.ts

```
/**
 * Agent Execution Engine
 * Core autonomous execution system that manages the full lifecycle
 * of task execution, from planning to completion.
 */
export class AgentExecutionEngine {
  // Main execution loop
  async executeTask(params: ExecuteTaskParams): Promise<ExecutionResult>

  // Sub-processes
  private async planExecution(taskId: string): Promise<ExecutionPlan>
  private async executeSteps(plan: ExecutionPlan): Promise<void>
  private async handleToolCall(tool: ToolCall): Promise<ToolResult>
  private async generateCode(spec: CodeSpec): Promise<GeneratedCode>
  private async buildProject(projectPath: string): Promise<BuildResult>
  private async deployPreview(buildPath: string): Promise<DeploymentInfo>
}
```

3.2 Execution Flow



3.3 State Management

The execution engine maintains state through PostgreSQL:

```

-- execution_state table (NEW)
CREATE TABLE execution_state (
  id UUID PRIMARY KEY,
  execution_id UUID REFERENCES executions(id),
  current_step TEXT,
  step_index INTEGER,
  total_steps INTEGER,
  context JSONB, -- Execution context data
  variables JSONB, -- Runtime variables
  artifacts JSONB, -- Generated artifacts
  created_at TIMESTAMPTZ,
  updated_at TIMESTAMPTZ
);

```

3.4 Implementation Details

File Location: `/lib/agents/execution-engine.ts`

Key Features:

1. **Autonomous Decision Making:** LLM-powered task planning and execution
2. **Tool Orchestration:** Dynamic tool selection and invocation
3. **Error Recovery:** Automatic retry with exponential backoff
4. **Progress Streaming:** Real-time event emission via Redis
5. **Resource Management:** Memory and CPU monitoring
6. **Cancellation Support:** Graceful shutdown on user request

Dependencies:

- Planning Service (existing)
- Tool Execution Service (existing)
- LLM Router (existing)
- Build Service (NEW)
- Deploy Service (NEW)
- Code Generator (NEW)

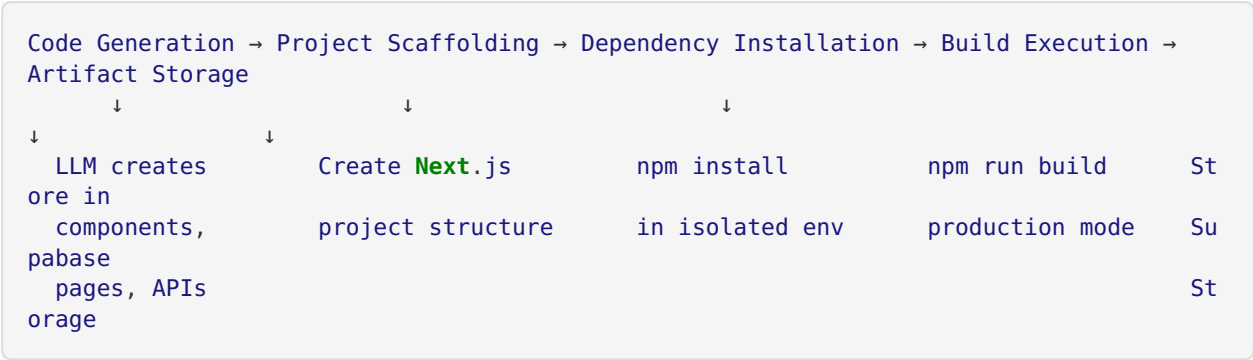
4. Build System Architecture

4.1 Overview

The build system handles the creation, configuration, and building of Next.js/React applications based on user requirements.

Location: `/lib/services/build-service.ts`

4.2 Build Pipeline



4.3 Build Service Implementation

```
/**
 * Build Service
 * Handles project generation, dependency management, and building
 */
export class BuildService {
  // Create new project
  async createProject(spec: ProjectSpec): Promise<Project>

  // Generate code files
  async generateFiles(project: Project, files: FileSpec[]): Promise<void>

  // Install dependencies
  async installDependencies(projectPath: string): Promise<void>

  // Run build
  async buildProject(projectPath: string): Promise<BuildResult>

  // Clean up build artifacts
  async cleanup(projectPath: string): Promise<void>
}
```

4.4 Project Template

The system will use a base Next.js template stored at:

Location: `/lib/templates/nextjs-base/`

```
/lib/templates/nextjs-base/
├── package.json           # Base dependencies
├── next.config.mjs        # Next.js config
├── tailwind.config.ts     # Styling config
├── tsconfig.json          # TypeScript config
├── app/
│   ├── layout.tsx         # Root layout
│   └── page.tsx           # Home page (template)
├── components/           # Base components
│   └── ui/                # shadcn components
└── public/               # Static assets
```

4.5 Build Isolation

Each build runs in an isolated environment to prevent conflicts:

Approach 1: Separate Directories

```
/tmp/builds/
├── run-{uuid}-{timestamp}/
│   ├── project/           # Generated project
│   ├── node_modules/     # Dependencies
│   └── .next/             # Build output
```

Approach 2: Docker Containers (Future)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build
```

4.6 Build Caching Strategy

To improve build times:

1. **Dependency Caching:** Cache `node_modules` for common dependency sets
2. **Template Caching:** Pre-built base templates
3. **Incremental Builds:** Only rebuild changed files
4. **Parallel Builds:** Support multiple concurrent builds

Cache Storage: Redis + File System

```
interface BuildCache {
  dependencyHash: string;
  nodeModulesPath: string;
  createdAt: Date;
  expiresAt: Date;
}
```

4.7 Build Events

The build service emits real-time events:

```
enum BuildEventType {
  BUILD_STARTED = 'BUILD_STARTED',
  DEPENDENCIES_INSTALLING = 'DEPENDENCIES_INSTALLING',
  DEPENDENCIES_INSTALLED = 'DEPENDENCIES_INSTALLED',
  BUILD_RUNNING = 'BUILD_RUNNING',
  BUILD_COMPLETED = 'BUILD_COMPLETED',
  BUILD_FAILED = 'BUILD_FAILED',
  BUILD_LOG = 'BUILD_LOG'
}
```

5. Deployment & Preview System

5.1 Overview

The deployment system provides isolated preview environments for generated applications.

Location: /lib/services/deploy-service.ts

5.2 Deployment Architecture

```
Build Output → Preview Server Start → Port Assignment → URL Generation → User Access
    ↓                ↓                ↓                ↓                ↓
.next dir      next start -p PORT  Find available  Create secure  Embed in
Static files   (production mode)  port (3000-4000)  preview URL   chat iframe
```

5.3 Preview Server Management

Option 1: Embedded Next.js Server (Recommended for MVP)

Pros:

- Simple implementation
- No additional infrastructure
- Fast startup

Cons:

- Limited scalability
- Resource constraints
- Security considerations

Implementation:


```

export class PreviewManager {
  private activeServers: Map<string, PreviewServer> = new Map();

  async startPreview(buildPath: string): Promise<PreviewInfo> {
    const port = await this.findAvailablePort();
    const process = spawn('npx', ['next', 'start', '-p', port], {
      cwd: buildPath
    });

    const previewId = uuidv4();
    const url = `http://localhost:${port}`;

    this.activeServers.set(previewId, {
      process,
      port,
      buildPath,
      startTime: Date.now()
    });

    return { previewId, url, port };
  }

  async stopPreview(previewId: string): Promise<void> {
    const server = this.activeServers.get(previewId);
    if (server) {
      server.process.kill();
      this.activeServers.delete(previewId);
    }
  }
}

```

Option 2: Docker Containers (Production)

Pros:

- Better isolation
- Resource limits
- Scalable

Cons:

- More complex setup
- Slower startup
- Requires Docker

Implementation:

```

async startPreview(buildPath: string): Promise<PreviewInfo> {
  const containerId = await docker.run({
    image: 'node:20-alpine',
    volumes: [`${buildPath}:/app`],
    ports: ['3000'],
    command: 'npm start',
    env: {
      NODE_ENV: 'production'
    }
  });

  const port = await docker.getPublishedPort(containerId, 3000);
  return { containerId, url: `http://localhost:${port}` };
}

```

5.4 URL Management

Database Schema for Previews:

```

-- preview_deployments table (NEW)
CREATE TABLE preview_deployments (
  id UUID PRIMARY KEY,
  run_id UUID REFERENCES runs(id),
  build_id UUID REFERENCES builds(id),
  preview_url TEXT NOT NULL,
  internal_port INTEGER NOT NULL,
  status TEXT NOT NULL, -- starting, running, stopped, failed
  process_id TEXT,
  started_at TIMESTAMP,
  stopped_at TIMESTAMP,
  last_accessed_at TIMESTAMP,
  metadata JSONB,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX preview_deployments_run_id_idx ON preview_deployments(run_id);
CREATE INDEX preview_deployments_status_idx ON preview_deployments(status);

```

5.5 Preview Lifecycle

1. **Start:** Build completes → Start preview server → Register in DB
2. **Monitor:** Health checks every 30s → Update last_accessed_at
3. **Auto-Stop:** After 1 hour of inactivity → Stop server → Update status
4. **Cleanup:** Delete files after 24 hours

5.6 Security & Isolation

Security Measures:

1. **Port Range Restriction:** Only use ports 3000-4000
2. **Process Isolation:** Run under limited user permissions
3. **Resource Limits:** CPU and memory caps
4. **Network Isolation:** No external network access
5. **Timeout:** Maximum runtime of 2 hours

Isolation Strategy:

```
const limits = {
  maxMemory: '512MB',
  maxCpu: '0.5',
  maxRuntime: 2 * 60 * 60 * 1000, // 2 hours
  networkAccess: false
};
```

6. Database Schema Extensions

6.1 New Tables Required

builds Table

Tracks build operations for generated projects.

```
CREATE TABLE builds (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  run_id UUID REFERENCES runs(id) ON DELETE CASCADE,
  execution_id UUID REFERENCES executions(id) ON DELETE CASCADE,
  user_id TEXT NOT NULL,
  project_name TEXT NOT NULL,
  project_type TEXT NOT NULL, -- nextjs, react, html
  status TEXT NOT NULL, -- queued, installing, building, completed, failed
  build_path TEXT, -- File system path
  output_path TEXT, -- Build output path (.next, dist, etc)
  build_logs TEXT, -- Build logs
  error_message TEXT,
  started_at TIMESTAMP,
  completed_at TIMESTAMP,
  duration_ms INTEGER,
  size_bytes BIGINT,
  metadata JSONB, -- Project specs, dependencies, etc
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX builds_run_id_idx ON builds(run_id);
CREATE INDEX builds_execution_id_idx ON builds(execution_id);
CREATE INDEX builds_user_id_idx ON builds(user_id);
CREATE INDEX builds_status_idx ON builds(status);
```

build_artifacts Table

Stores generated code files and assets.

```

CREATE TABLE build_artifacts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  build_id UUID REFERENCES builds(id) ON DELETE CASCADE,
  file_path TEXT NOT NULL, -- Relative path in project
  file_type TEXT NOT NULL, -- tsx, ts, css, json, etc
  content TEXT, -- File content (for small files)
  storage_path TEXT, -- Supabase storage path (for large files)
  size_bytes INTEGER,
  mime_type TEXT,
  is_generated BOOLEAN DEFAULT TRUE,
  metadata JSONB,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX build_artifacts_build_id_idx ON build_artifacts(build_id);
CREATE INDEX build_artifacts_file_type_idx ON build_artifacts(file_type);

```

code_generations Table

Tracks individual code generation requests.

```

CREATE TABLE code_generations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  run_id UUID REFERENCES runs(id) ON DELETE CASCADE,
  subtask_id UUID REFERENCES run_subtasks(id) ON DELETE SET NULL,
  user_id TEXT NOT NULL,
  prompt TEXT NOT NULL, -- What to generate
  generated_code TEXT NOT NULL,
  language TEXT NOT NULL, -- typescript, javascript, css, etc
  framework TEXT, -- react, nextjs, etc
  llm_provider TEXT NOT NULL,
  llm_model TEXT NOT NULL,
  tokens_used INTEGER,
  generation_time_ms INTEGER,
  validation_status TEXT, -- valid, invalid, unchecked
  validation_errors JSONB,
  metadata JSONB,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX code_generations_run_id_idx ON code_generations(run_id);
CREATE INDEX code_generations_user_id_idx ON code_generations(user_id);

```

execution_state Table

Stores agent execution state for recovery and debugging.

```
CREATE TABLE execution_state (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  execution_id UUID REFERENCES executions(id) ON DELETE CASCADE,
  run_id UUID REFERENCES runs(id) ON DELETE CASCADE,
  current_step TEXT NOT NULL,
  step_index INTEGER NOT NULL,
  total_steps INTEGER NOT NULL,
  context JSONB, -- Execution context variables
  variables JSONB, -- Runtime variables
  artifacts JSONB, -- Generated artifacts references
  decisions JSONB[], -- Agent decision history
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX execution_state_execution_id_idx ON execution_state(execution_id);
CREATE INDEX execution_state_run_id_idx ON execution_state(run_id);
```

6.2 Schema Migration

Migration File: /db/migrations/0005_agent_execution_system.sql

```
-- Add all new tables from above
-- Add new columns to existing tables (if needed)

-- Example: Add build_id to run_artifacts
ALTER TABLE run_artifacts
ADD COLUMN build_id UUID REFERENCES builds(id) ON DELETE CASCADE;

CREATE INDEX run_artifacts_build_id_idx ON run_artifacts(build_id);
```

6.3 Drizzle Schema Files

Create new schema files:

- /db/schema/builds.ts
- /db/schema/code-generations.ts
- /db/schema/execution-state.ts

Update /db/schema/index.ts :

```
export * from "./builds";
export * from "./code-generations";
export * from "./execution-state";
// ... existing exports
```

7. API Route Structure

7.1 New API Routes

Agent Execution Routes

Location: /app/api/agent/

Route	Method	Purpose
/api/agent/execute	POST	Start agent execution
/api/agent/[executionId]/status	GET	Get execution status
/api/agent/[executionId]/cancel	POST	Cancel execution
/api/agent/[executionId]/retry	POST	Retry failed execution

Build Routes

Location: /app/api/builds/

Route	Method	Purpose
/api/builds/create	POST	Create new build
/api/builds/[buildId]	GET	Get build details
/api/builds/[buildId]/logs	GET	Stream build logs (SSE)
/api/builds/[buildId]/artifacts	GET	List build artifacts
/api/builds/[buildId]/download	GET	Download build archive

Preview Routes

Location: /app/api/previews/

Route	Method	Purpose
/api/previews/start	POST	Start preview deployment
/api/previews/[previewId]	GET	Get preview info
/api/previews/[previewId]/stop	POST	Stop preview
/api/previews/[previewId]/restart	POST	Restart preview
/api/previews/proxy	GET	Proxy to preview (if needed)

Code Generation Routes

Location: `/app/api/code/`

Route	Method	Purpose
<code>/api/code/generate</code>	POST	Generate code snippet
<code>/api/code/[generationId]</code>	GET	Get generated code
<code>/api/code/validate</code>	POST	Validate generated code
<code>/api/code/stream</code>	GET	Stream code generation (SSE)

7.2 API Implementations

Example: Execute Agent

File: `/app/api/agent/execute/route.ts`

```

import { NextRequest, NextResponse } from 'next/server';
import { auth } from '@clerk/nextjs/server';
import { AgentExecutionEngine } from '@lib/agents/execution-engine';
import { db } from '@db';
import { executionsTable, runsTable } from '@db/schema';

export async function POST(request: NextRequest) {
  const { userId } = await auth();
  if (!userId) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  const { prompt, taskId } = await request.json();

  // Create execution record
  const [execution] = await db.insert(executionsTable).values({
    taskId,
    status: 'queued',
    startTime: new Date()
  }).returning();

  // Create run record
  const [run] = await db.insert(runsTable).values({
    executionId: execution.id,
    userId,
    status: 'queued',
    title: prompt,
    totalSteps: 0
  }).returning();

  // Start async execution
  const engine = new AgentExecutionEngine();
  engine.executeTask({
    executionId: execution.id,
    runId: run.id,
    userId,
    prompt
  }).catch(error => {
    console.error('Agent execution failed:', error);
  });

  return NextResponse.json({
    success: true,
    executionId: execution.id,
    runId: run.id
  }, { status: 201 });
}

```

Example: Stream Build Logs

File: /app/api/builds/[buildId]/logs/route.ts


```
import { NextRequest } from 'next/server';
import { subscribeToChannel, getBuildChannel } from '@lib/redis';

export async function GET(
  request: NextRequest,
  { params }: { params: Promise<{ buildId: string }> }
) {
  const { buildId } = await params;

  const stream = new ReadableStream({
    async start(controller) {
      const encoder = new TextEncoder();
      const channel = getBuildChannel(buildId);

      const subscriber = await subscribeToChannel(channel, (message) => {
        const event = `data: ${JSON.stringify(message)}\n\n`;
        controller.enqueue(encoder.encode(event));
      });

      // Cleanup on disconnect
      request.signal.addEventListener('abort', async () => {
        await subscriber.quit();
        controller.close();
      });
    }
  });

  return new Response(stream, {
    headers: {
      'Content-Type': 'text/event-stream',
      'Cache-Control': 'no-cache',
      'Connection': 'keep-alive'
    }
  });
}
```

8. WebSocket Integration

8.1 Real-Time Communication Architecture

The system uses **Server-Sent Events (SSE)** for real-time updates (already implemented in `/app/api/streams/runs/[runId]/route.ts`).

8.2 Event Types

Agent Execution Events

```
enum AgentEventType {  
  // Execution lifecycle  
  EXECUTION_STARTED = 'EXECUTION_STARTED',  
  EXECUTION_PROGRESS = 'EXECUTION_PROGRESS',  
  EXECUTION_COMPLETED = 'EXECUTION_COMPLETED',  
  EXECUTION_FAILED = 'EXECUTION_FAILED',  
  
  // Step events  
  STEP_STARTED = 'STEP_STARTED',  
  STEP_PROGRESS = 'STEP_PROGRESS',  
  STEP_COMPLETED = 'STEP_COMPLETED',  
  
  // Tool events  
  TOOL_CALLED = 'TOOL_CALLED',  
  TOOL_RESULT = 'TOOL_RESULT',  
  
  // Code generation events  
  CODE_GENERATION_STARTED = 'CODE_GENERATION_STARTED',  
  CODE_GENERATION_CHUNK = 'CODE_GENERATION_CHUNK',  
  CODE_GENERATION_COMPLETED = 'CODE_GENERATION_COMPLETED',  
  
  // Build events  
  BUILD_STARTED = 'BUILD_STARTED',  
  BUILD_LOG = 'BUILD_LOG',  
  BUILD_COMPLETED = 'BUILD_COMPLETED',  
  BUILD_FAILED = 'BUILD_FAILED',  
  
  // Preview events  
  PREVIEW_STARTING = 'PREVIEW_STARTING',  
  PREVIEW_READY = 'PREVIEW_READY',  
  PREVIEW_FAILED = 'PREVIEW_FAILED'  
}
```

8.3 Event Publishing

Service: `/lib/services/event-publisher.ts`

```

import { getRedisPublisher, getRunChannel } from '@lib/redis';
import { db } from '@db';
import { runEventsTable } from '@db/schema';

export class EventPublisher {
  static async publishEvent(params: {
    runId: string;
    subtaskId?: string;
    eventType: string;
    message: string;
    data?: any;
  }) {
    const { runId, subtaskId, eventType, message, data } = params;

    // Store in database for replay
    const [event] = await db.insert(runEventsTable).values({
      runId,
      subtaskId,
      eventType,
      message,
      data
    }).returning();

    // Publish to Redis for real-time streaming
    const redis = await getRedisPublisher();
    const channel = getRunChannel(runId);

    await redis.publish(channel, JSON.stringify({
      id: event.id,
      runId,
      subtaskId,
      eventType,
      message,
      data,
      timestamp: event.timestamp.toISOString()
    }));
  }

  static async publishCodeChunk(runId: string, chunk: string) {
    await this.publishEvent({
      runId,
      eventType: 'CODE_GENERATION_CHUNK',
      message: 'Code chunk generated',
      data: { chunk }
    });
  }

  static async publishBuildLog(runId: string, log: string) {
    await this.publishEvent({
      runId,
      eventType: 'BUILD_LOG',
      message: log,
      data: { log }
    });
  }
}

```

8.4 Frontend Event Consumption

The existing `useRunStream` hook (at `/components/progress/use-run-stream.ts`) will be extended to handle new event types:

```
// Extension to existing hook
switch (event.eventType) {
  case 'CODE_GENERATION_CHUNK':
    // Append code chunk to state
    setCodeBuffer(prev => prev + event.data.chunk);
    break;

  case 'BUILD_LOG':
    // Add build log
    setBuildLogs(prev => [...prev, event.data.log]);
    break;

  case 'PREVIEW_READY':
    // Set preview URL
    setPreviewUrl(event.data.url);
    break;
}
```

9. Tool Framework Architecture

9.1 Extended Tool System

The existing tool framework will be extended with new tools for code generation and building.

9.2 New Tools

Code Generation Tool

File: /lib/tools/code-generator.ts

```
export async function executeCodeGeneration(input: {
  prompt: string;
  type: 'component' | 'page' | 'api' | 'utility';
  framework: 'react' | 'nextjs';
  language: 'typescript' | 'javascript';
}): Promise<{ code: string; language: string }> {
  // Use LLM to generate code
  const response = await routeAndExecute({
    prompt: buildCodeGenerationPrompt(input),
    system: CODE_GENERATION_SYSTEM_PROMPT,
    context: {
      taskType: 'code',
      complexity: 'medium',
      maxTokens: 4000
    }
  });

  return {
    code: extractCode(response.content),
    language: input.language
  };
}
```

File Operations Tool

File: /lib/tools/file-operations.ts

```
export async function executeFileOperation(input: {
  operation: 'create' | 'read' | 'update' | 'delete';
  path: string;
  content?: string;
  projectPath: string;
}): Promise<{ success: boolean; content?: string }> {
  const fullPath = join(input.projectPath, input.path);

  switch (input.operation) {
    case 'create':
      await writeFile(fullPath, input.content || '');
      return { success: true };

    case 'read':
      const content = await readFile(fullPath, 'utf-8');
      return { success: true, content };

    case 'update':
      await writeFile(fullPath, input.content || '');
      return { success: true };

    case 'delete':
      await unlink(fullPath);
      return { success: true };
  }
}
```

Build Tool

File: /lib/tools/build-tool.ts

```

export async function executeBuild(input: {
  projectPath: string;
  buildCommand?: string;
}): Promise<{ success: boolean; output: string; error?: string }> {
  const buildCommand = input.buildCommand || 'npm run build';

  return new Promise((resolve) => {
    const process = spawn(buildCommand, [], {
      cwd: input.projectPath,
      shell: true
    });

    let output = '';
    let error = '';

    process.stdout.on('data', (data) => {
      output += data.toString();
    });

    process.stderr.on('data', (data) => {
      error += data.toString();
    });

    process.on('close', (code) => {
      resolve({
        success: code === 0,
        output,
        error: code !== 0 ? error : undefined
      });
    });
  });
}

```

9.3 Tool Registry Updates

Update the tool registry to include new tools:

File: /scripts/seed-tools.ts

```
const newTools = [
  {
    key: 'code_generator',
    name: 'Code Generator',
    version: '1.0.0',
    description: 'Generates code using LLM',
    manifest: {
      inputSchema: { /* ... */ },
      outputSchema: { /* ... */ },
      resources: {
        timeoutSec: 60,
        memMb: 512
      }
    }
  },
  {
    key: 'file_operations',
    name: 'File Operations',
    version: '1.0.0',
    description: 'Create, read, update, delete files',
    manifest: { /* ... */ }
  },
  {
    key: 'build_tool',
    name: 'Build Tool',
    version: '1.0.0',
    description: 'Builds Next.js/React projects',
    manifest: { /* ... */ }
  }
];
```

10. Frontend Integration Points

10.1 Chat Interface Enhancements

File: /components/chat/chat-interface.tsx

New Features:

1. **Code Block Rendering:** Syntax highlighting for generated code
2. **Preview Embedding:** Iframe for live previews
3. **Build Progress:** Visual build status indicators
4. **Artifact Links:** Download buttons for generated files

```

import { Prism as SyntaxHighlighter } from 'react-syntax-highlighter';
import { PreviewFrame } from './preview-frame';
import { BuildProgress } from './build-progress';

function ChatInterface() {
  const { runState, isConnected } = useRunStream({ runId });

  return (
    <div className="chat-container">
      <MessageList messages={messages} />

      { /* Code generation display */ }
      {runState.currentCode && (
        <SyntaxHighlighter language="typescript">
          {runState.currentCode}
        </SyntaxHighlighter>
      )}

      { /* Build progress */ }
      {runState.buildStatus && (
        <BuildProgress status={runState.buildStatus} logs={runState.buildLogs} />
      )}

      { /* Preview frame */ }
      {runState.previewUrl && (
        <PreviewFrame url={runState.previewUrl} />
      )}

      <MessageInput onSend={handleSendMessage} />
    </div>
  );
}

```

10.2 Preview Frame Component

File: /components/chat/preview-frame.tsx


```

'use client';

interface PreviewFrameProps {
  url: string;
  title?: string;
}

export function PreviewFrame({ url, title }: PreviewFrameProps) {
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  return (
    <div className="preview-container">
      <div className="preview-header">
        <h3>{title || 'Preview'}</h3>
        <div className="preview-actions">
          <Button onClick={() => window.open(url, '_blank')}>
            Open in New Tab
          </Button>
        </div>
      </div>
      <div>
        <div className="preview-frame">
          {isLoading && <div className="loading">Loading preview...</div>}
          {error && <div className="error">{error}</div>}

          <iframe
            src={url}
            width="100%"
            height="600px"
            sandbox="allow-scripts allow-same-origin"
            onLoad={() => setIsLoading(false)}
            onError={() => setError('Failed to load preview')}
          />
        </div>
      </div>
    </div>
  );
}

```

10.3 Build Progress Component

File: /components/chat/build-progress.tsx

```
'use client';

interface BuildProgressProps {
  status: 'installing' | 'building' | 'completed' | 'failed';
  logs: string[];
}

export function BuildProgress({ status, logs }: BuildProgressProps) {
  return (
    <div className="build-progress">
      <div className="build-status">
        {status === 'installing' && (
          <div className="status-badge installing">Installing dependencies...</div>
        )}
        {status === 'building' && (
          <div className="status-badge building">Building project...</div>
        )}
        {status === 'completed' && (
          <div className="status-badge completed">Build completed!</div>
        )}
        {status === 'failed' && (
          <div className="status-badge failed">Build failed</div>
        )}
      </div>

      <div className="build-logs">
        <h4>Build Logs</h4>
        <pre className="logs-container">
          {logs.map((log, i) => (
            <div key={i} className="log-line">{log}</div>
          ))}
        </pre>
      </div>
    </div>
  );
}
```

10.4 Run Progress Panel Updates

Extend the existing run progress panel at `/components/runs/run-progress-panel.tsx` to show:

- Code generation progress
- File creation events
- Build status
- Preview availability

11. File & Artifact Management

11.1 File Storage Strategy

Temporary Files (Build artifacts, work in progress)

- **Location:** `/tmp/builds/{runId}/`
- **Retention:** 24 hours
- **Cleanup:** Automatic via cron job

Permanent Files (Completed projects, user artifacts)

- **Location:** Supabase Storage

- **Bucket:** `mindous-artifacts`
- **Structure:** `{userId}/{runId}/{filename}`
- **Retention:** 30 days (configurable per user plan)

11.2 File Manager Service

File: `/lib/services/file-manager.ts`

```

import { createClient } from '@supabase/supabase-js';
import { writeFile, readFile, mkdir, rm } from 'fs/promises';
import { join } from 'path';

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!
);

export class FileManager {
  /**
   * Create temporary build directory
   */
  static async createBuildDir(runId: string): Promise<string> {
    const buildPath = join('/tmp/builds', runId);
    await mkdir(buildPath, { recursive: true });
    return buildPath;
  }

  /**
   * Save file to temporary storage
   */
  static async saveTempFile(
    runId: string,
    relativePath: string,
    content: string
  ): Promise<string> {
    const buildPath = await this.createBuildDir(runId);
    const fullPath = join(buildPath, relativePath);

    // Create parent directories
    const dir = dirname(fullPath);
    await mkdir(dir, { recursive: true });

    await writeFile(fullPath, content, 'utf-8');
    return fullPath;
  }

  /**
   * Upload to Supabase Storage
   */
  static async uploadToStorage(
    userId: string,
    runId: string,
    filename: string,
    content: Buffer | string
  ): Promise<string> {
    const path = `${userId}/${runId}/${filename}`;

    const { data, error } = await supabase.storage
      .from('mindous-artifacts')
      .upload(path, content, {
        contentType: this.getContentType(filename),
        upsert: true
      });

    if (error) throw error;

    return data.path;
  }
}

```

```

    * Download from Supabase Storage
    */
    static async downloadFromStorage(path: string): Promise<Buffer> {
        const { data, error } = await supabase.storage
            .from('mindous-artifacts')
            .download(path);

        if (error) throw error;

        return Buffer.from(await data.arrayBuffer());
    }

    /**
     * Clean up temporary files
     */
    static async cleanupBuildDir(runId: string): Promise<void> {
        const buildPath = join('/tmp/builds', runId);
        await rm(buildPath, { recursive: true, force: true });
    }

    /**
     * Archive build directory to tar.gz
     */
    static async archiveBuild(runId: string): Promise<string> {
        const buildPath = join('/tmp/builds', runId);
        const archivePath = join('/tmp/builds', `${runId}.tar.gz`);

        // Use tar command
        const { spawn } = require('child_process');

        return new Promise((resolve, reject) => {
            const process = spawn('tar', ['-czf', archivePath, '-C', buildPath, '.']);

            process.on('close', (code: number) => {
                if (code === 0) resolve(archivePath);
                else reject(new Error(`tar failed with code ${code}`));
            });
        });
    }

    private static getContentType(filename: string): string {
        const ext = filename.split('.').pop()?.toLowerCase();
        const types: Record<string, string> = {
            'ts': 'text/typescript',
            'tsx': 'text/typescript',
            'js': 'text/javascript',
            'jsx': 'text/javascript',
            'json': 'application/json',
            'css': 'text/css',
            'html': 'text/html',
            'md': 'text/markdown'
        };
        return types[ext || ''] || 'text/plain';
    }
}

```

11.3 Artifact Storage in Database

Store metadata about artifacts in the database:

```
// When saving artifact
await db.insert(runArtifactsTable).values({
  runId,
  subtaskId,
  name: filename,
  type: 'code',
  path: storagePath, // Supabase storage path
  size: content.length,
  mimeType: FileManager.getContentType(filename),
  metadata: {
    language: 'typescript',
    framework: 'nextjs',
    generated: true
  }
});
```

11.4 Cleanup Jobs

Cron Job: Clean up old temporary files

File: /lib/jobs/cleanup-builds.ts

```
import { rm, readdir } from 'fs/promises';
import { join } from 'path';

export async function cleanupOldBuilds() {
  const buildsDir = '/tmp/builds';
  const entries = await readdir(buildsDir, { withFileTypes: true });

  const now = Date.now();
  const maxAge = 24 * 60 * 60 * 1000; // 24 hours

  for (const entry of entries) {
    if (entry.isDirectory()) {
      const path = join(buildsDir, entry.name);
      const stat = await stat(path);

      if (now - stat.mtimeMs > maxAge) {
        console.log(`Cleaning up old build: ${entry.name}`);
        await rm(path, { recursive: true, force: true });
      }
    }
  }
}

// Run every hour
setInterval(cleanupOldBuilds, 60 * 60 * 1000);
```

12. Implementation Roadmap

12.1 Phase 1: Foundation (Week 1-2)

Priority: HIGH

1. Database Schema

- [] Create migration for new tables (builds, code_generations, execution_state, pre-

view_deployments)

- [] Add Drizzle schema files
- [] Run migrations
- [] Test schema with sample data

2. Build Service

- [] Implement basic BuildService class
- [] Create Next.js project template
- [] Add project scaffolding logic
- [] Implement dependency installation
- [] Add build execution

3. File Manager

- [] Implement FileManager service
- [] Set up Supabase storage bucket
- [] Add temp file operations
- [] Implement cleanup logic

Deliverable: Basic build system that can create and build a simple Next.js project

12.2 Phase 2: Code Generation (Week 3-4)

Priority: HIGH

1. Code Generator Service

- [] Implement CodeGenerator class
- [] Create code generation prompts
- [] Add component generation
- [] Add page generation
- [] Add API route generation

2. Code Generation Tool

- [] Create code_generator tool
- [] Add to tool registry
- [] Implement validation

3. API Routes

- [] /api/code/generate endpoint
- [] /api/code/stream for streaming
- [] /api/code/validate for validation

Deliverable: Code generation system that can create React components

12.3 Phase 3: Agent Engine (Week 5-6)

Priority: CRITICAL

1. Agent Execution Engine

- [] Implement AgentExecutionEngine class
- [] Add execution loop logic
- [] Integrate planning service
- [] Add tool orchestration
- [] Implement state management

2. Execution State Management

- [] Implement state persistence
- [] Add recovery logic
- [] Add execution history

3. API Routes

- [] `/api/agent/execute` endpoint
- [] `/api/agent/[executionId]/status`
- [] `/api/agent/[executionId]/cancel`

Deliverable: Autonomous agent that can execute multi-step tasks

12.4 Phase 4: Preview System (Week 7-8)

Priority: HIGH

1. Preview Manager

- [] Implement PreviewManager class
- [] Add port management
- [] Implement server lifecycle
- [] Add health monitoring

2. Deploy Service

- [] Implement DeployService class
- [] Add preview URL generation
- [] Implement auto-cleanup

3. API Routes

- [] `/api/previews/start` endpoint
- [] `/api/previews/[previewId]` endpoint
- [] `/api/previews/[previewId]/stop`

4. Frontend Components

- [] Create PreviewFrame component
- [] Integrate in chat interface
- [] Add preview controls

Deliverable: Working preview system with iframe embedding

12.5 Phase 5: Integration & Polish (Week 9-10)

Priority: MEDIUM

1. Event Streaming

- [] Extend event types
- [] Add code streaming
- [] Add build log streaming
- [] Update frontend hooks

2. Frontend Integration

- [] Add BuildProgress component
- [] Add code highlighting
- [] Update chat interface
- [] Add artifact download links

3. Testing & Debugging

- [] End-to-end testing
- [] Error handling improvements
- [] Performance optimization
- [] Bug fixes

Deliverable: Fully integrated system ready for testing

12.6 Phase 6: Production Ready (Week 11-12)

Priority: MEDIUM

1. Security Hardening

- [] Add rate limiting
- [] Implement resource limits
- [] Add input validation
- [] Security audit

2. Performance Optimization

- [] Add build caching
- [] Optimize database queries
- [] Add Redis caching
- [] Load testing

3. Documentation

- [] API documentation
- [] User guides
- [] Developer documentation
- [] Deployment guide

Deliverable: Production-ready agent execution system

13. Security & Permissions

13.1 Security Considerations

Code Execution Isolation

1. **Process Isolation:** Run build processes with limited permissions
2. **Resource Limits:** CPU and memory caps per build
3. **Timeout Enforcement:** Maximum execution time
4. **Network Restrictions:** No outbound connections during build
5. **File System Restrictions:** Limited to build directory only

Preview Isolation

1. **Port Range:** Restrict to 3000-4000 range
2. **Process User:** Run under non-root user
3. **Memory Limits:** Maximum 512MB per preview
4. **Timeout:** Auto-stop after 2 hours
5. **Sandbox:** Iframe sandbox attributes

Input Validation

- 1. **Prompt Sanitization:** Remove malicious code patterns
- 2. **File Path Validation:** Prevent path traversal
- 3. **Code Validation:** AST parsing before execution
- 4. **Dependency Validation:** Whitelist allowed npm packages

13.2 Rate Limiting

API Rate Limits:

Endpoint	Limit	Window
/api/agent/execute	10 requests	1 hour
/api/code/generate	50 requests	1 hour
/api/builds/create	20 requests	1 hour
/api/previews/start	10 requests	1 hour

Implementation:

```
import { Ratelimit } from '@upstash/ratelimit';
import { Redis } from '@upstash/redis';

const ratelimit = new Ratelimit({
  redis: Redis.fromEnv(),
  limiter: Ratelimit.slidingWindow(10, '1 h')
});

export async function checkRateLimit(userId: string) {
  const { success, limit, remaining } = await ratelimit.limit(userId);

  if (!success) {
    throw new Error('Rate limit exceeded');
  }

  return { limit, remaining };
}
```

13.3 Resource Quotas

Per-user quotas (based on plan):

Resource	Free Plan	Pro Plan	Enterprise
Builds/month	10	100	Unlimited
Storage	500MB	10GB	Custom
Preview uptime	1 hour	12 hours	24 hours
LLM tokens/month	100K	1M	Custom

Implementation:

```
export async function checkQuota(
  userId: string,
  resource: 'builds' | 'storage' | 'tokens'
): Promise<{ allowed: boolean; remaining: number }> {
  const usage = await getUserUsage(userId, resource);
  const plan = await getUserPlan(userId);
  const limit = PLAN_LIMITS[plan][resource];

  return {
    allowed: usage < limit,
    remaining: limit - usage
  };
}
```

14. Performance & Scalability

14.1 Performance Optimization

Build Performance

1. **Dependency Caching:** Cache `node_modules` for common dependency sets
 - Cache key: Hash of `package.json` dependencies
 - Storage: Redis + File system
 - TTL: 7 days
2. **Template Pre-warming:** Pre-built base templates
 - Keep 3-5 pre-built Next.js templates ready
 - Reduces build time from 2-3 minutes to 30 seconds
3. **Parallel Builds:** Support multiple concurrent builds
 - Max concurrent builds: 5 (configurable)
 - Queue excess requests

Code Generation Performance

1. **LLM Caching:** Cache common code patterns
 - Use existing LLM router cache
 - TTL: 1 hour for code generation
2. **Streaming:** Stream code generation to user
 - Better perceived performance
 - User sees progress immediately

Preview Performance

1. **Server Pooling:** Keep 2-3 preview servers warm
 - Reduces startup time
 - Better user experience
2. **Port Reuse:** Reuse ports after cleanup
 - Efficient resource usage

14.2 Scalability Strategy

Horizontal Scaling

1. **Stateless Design:** All state in PostgreSQL/Redis
2. **Load Balancing:** Multiple Next.js instances
3. **Build Workers:** Separate build worker nodes

Vertical Scaling

1. **Resource Allocation:** More CPU/memory for build workers
2. **Database:** Scale Supabase instance as needed

Future Enhancements

1. **Docker Orchestration:** Kubernetes for build isolation
 2. **CDN:** CloudFlare for preview assets
 3. **Queue System:** Bull/BullMQ for build queue
 4. **Microservices:** Separate build service
-

Conclusion

This technical architecture document provides a comprehensive blueprint for implementing the agent execution engine in Mindous.ai. The design builds upon the existing infrastructure while adding the necessary components for autonomous app building, real-time streaming, and preview deployment.

Key Takeaways:

1. **Modular Design:** Each component is independent and can be developed/tested separately
2. **Incremental Implementation:** Phased approach allows for iterative development
3. **Scalability:** Architecture supports growth from MVP to production
4. **Security First:** Built-in security and isolation mechanisms
5. **Real-time Experience:** WebSocket/SSE for live updates

Next Steps:

1. Review and approve this architecture
 2. Set up development environment
 3. Begin Phase 1 implementation
 4. Establish testing procedures
 5. Create CI/CD pipeline
-

Appendix A: Key File Locations Summary

Core Services:

- ☐ /lib/agents/execution-engine.ts (NEW)
- ☐ /lib/services/build-service.ts (NEW)
- ☐ /lib/services/deploy-service.ts (NEW)
- ☐ /lib/services/code-generator.ts (NEW)
- ☐ /lib/services/file-manager.ts (NEW)
- ☐ /lib/services/preview-manager.ts (NEW)
- ☐ /lib/services/event-publisher.ts (NEW)

API Routes:

- ☐ /app/api/agent/ (NEW)
- ☐ /app/api/builds/ (NEW)
- ☐ /app/api/previews/ (NEW)
- ☐ /app/api/code/ (NEW)

Database:

- ☐ /db/schema/builds.ts (NEW)
- ☐ /db/schema/code-generations.ts (NEW)
- ☐ /db/schema/execution-state.ts (NEW)
- ☐ /db/migrations/0005_agent_execution_system.sql (NEW)

Frontend:

- ☐ /components/chat/preview-frame.tsx (NEW)
- ☐ /components/chat/build-progress.tsx (NEW)
- ☐ /components/chat/chat-interface.tsx (UPDATE)

Tools:

- ☐ /lib/tools/code-generator.ts (NEW)
- ☐ /lib/tools/file-operations.ts (NEW)
- ☐ /lib/tools/build-**tool**.ts (NEW)

Document End